

SQL to pandas cheatsheet

From the prAxIs Data Engineering stream, Notebooks 2 and 3. survey, other, and df stand for any table or dataframe.

Part 1: the core of SQL (Notebook 2)

SQL	pandas
SELECT col1, col2 FROM survey	survey[["col1", "col2"]]
WHERE hourly_wage >= 40	survey[survey["hourly_wage"] >= 40]
WHERE province IN (...)	survey[survey["province"].isin(...)]
SELECT DISTINCT province	survey["province"].drop_duplicates()
ORDER BY hourly_wage DESC	survey.sort_values("hourly_wage", ascending=False)
LIMIT 5	survey.head(5)
JOIN ... ON	survey.merge(other, on="key", how="inner")
LEFT JOIN ... ON	survey.merge(other, on="key", how="left")
COUNT(*), AVG(x)	len(survey), survey["x"].mean()
COUNT(DISTINCT x)	survey["x"].nunique()
GROUP BY g with aggregates	survey.groupby("g")["x"].mean()
HAVING COUNT(*) >= 50	survey.groupby("g").filter(lambda d: len(d) >= 50)
IS NULL	survey["x"].isna()
CASE WHEN ... END	np.where(condition, a, b) or pd.cut

Part 2: advanced SQL (Notebook 3)

SQL	pandas
WHERE id IN (SELECT id FROM other)	survey[survey["id"].isin(other["id"])]
derived table FROM (SELECT ...)	an intermediate dataframe you assign and reuse
WITH step AS (...) SELECT ... FROM step	named steps: step = ..., then work on step
RANK() OVER (PARTITION BY g ORDER BY x DESC)	df.groupby("g") ["x"].rank(ascending=False)
AVG(x) OVER (PARTITION BY g)	df.groupby("g")["x"].transform("mean")
SUM(x) OVER (ORDER BY x)	df.sort_values("x")["x"].cumsum()
LAG(x) OVER (PARTITION BY g ORDER BY t)	df.sort_values("t").groupby("g") ["x"].shift(1)
CREATE VIEW v AS ...	no direct equivalent: a saved query, living in the database
INSERT INTO t VALUES (...)	pd.concat([df, new_rows])
UPDATE t SET col = v WHERE cond	df.loc[cond, "col"] = v
DELETE FROM t WHERE cond	df = df[~cond]
COALESCE(x, 0)	df["x"].fillna(0)
? placeholders with params	pd.read_sql(sql, conn, params=(...))

The rules that matter more than syntax

1. Read your row counts: joins shrink, keep, or multiply.
2. Know what one row stands for before you aggregate.
3. WHERE runs before groups exist; HAVING runs after.
4. = NULL is never true; write IS NULL.
5. Write the SELECT first, then convert it to an UPDATE or DELETE.
6. SQL and data travel separately: ? placeholders, always.